Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Матвиенко Е.Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.10.26

Москва, 2026

Постановка задачи

Вариант 10.

Решить систему линейных уравнений (СЛАУ) методом Гаусса в многопоточном режиме с использованием стандартных средств создания и синхронизации потоков операционной системы (POSIX Threads). Программа должна позволять ограничивать максимальное количество одновременно работающих потоков с помощью аргументов командной строки.

Общий метод и алгоритм решения

В лабораторной работе реализуется параллельная версия метода Гаусса для решения СЛАУ на языке Си. Для управления потоками и их синхронизации используются низкоуровневые средства POSIX Threads (pthreads).

Использованные системные вызовы и функции потоков ОС:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg); - создает новый поток исполнения.
- int pthread_join(pthread_t thread, void **retval); - ожидает завершения указанного потока.
- int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count); и pthread_barrier_wait - инициализируют барьер и синхронизируют потоки на каждой итерации внешнего цикла.
- int pthread_barrier_destroy(pthread_barrier_t *barrier); - уничтожает барьер по завершении работы.
- int clock_gettime(clockid_t clk_id, struct timespec *tp); - точный замер времени выполнения в монотонном режиме.

Описание работы алгоритма:

1. Программа принимает размер матрицы N и количество потоков через аргументы командной строки с проверкой корректности ввода через strtol.
2. Генерируется расширенная матрица коэффициентов [A|b] псевдослучайными числами.
3. Выполняется последовательный алгоритм метода Гаусса (прямой ход с приведением к треугольному виду и обратный ход) для получения базового времени T_{seq}.
4. Выполняется параллельная версия: на каждом шаге прямого хода строки матрицы динамически распределяются между заданным количеством потоков. Потоки параллельно обнуляют элементы под главной диагональю, после чего синхронизируются через барьер. Обратный ход выполняется последовательно.
5. Рассчитываются метрики ускорения $(S = T_{seq}/T_{par})$ и эффективности $E = S/N$.

Код программы

gaussian_threads.c

```
    #include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef struct {
    int thread_id;
    int num_threads;
    int n;
    double **a;
    int k;
    pthread_barrier_t *barrier;
} thread_data_t;

static void* eliminate_rows(void *arg) {
    thread_data_t *data = (thread_data_t*) arg;
    int n = data->n;
    int k = data->k;
    double **a = data->a;
    int id = data->thread_id;
    int num_threads = data->num_threads;

    int total_rows = n - (k + 1);
    if (total_rows > 0) {
        int rows_per_thread = total_rows / num_threads;
        int remainder = total_rows % num_threads;

        int start = (k + 1) + id * rows_per_thread + (id < remainder ? id :
remainder);
        int end = start + rows_per_thread + (id < remainder ? 1 : 0);

        for (int i = start; i < end; i++) {
            double factor = a[i][k] / a[k][k];
            for (int j = k; j <= n; j++) {
                a[i][j] -= factor * a[k][j];
            }
        }
    }

    pthread_barrier_wait(data->barrier);
    return NULL;
}
```

```c
static double** allocate_matrix(int n) {
    double **a = malloc(n * sizeof(double*));
    for (int i = 0; i < n; i++) {
        a[i] = malloc((n + 1) * sizeof(double));
    }
    return a;
}

static void free_matrix(double **a, int n) {
    for (int i = 0; i < n; i++) free(a[i]);
    free(a);
}

static void generate_matrix(double **a, int n) {
    unsigned short xsubi[3] = {42, 42, 42};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= n; j++) {
            a[i][j] = erand48(xsubi) * 10.0 + 1.0;
        }
    }
}

static void sequential_gaussian(double **a, int n, double *x) {
    for (int k = 0; k < n - 1; k++) {
        for (int i = k + 1; i < n; i++) {
            double factor = a[i][k] / a[k][k];
            for (int j = k; j <= n; j++) {
                a[i][j] -= factor * a[k][j];
            }
        }
    }

    for (int i = n - 1; i >= 0; i--) {
        x[i] = a[i][n];
        for (int j = i + 1; j < n; j++) {
            x[i] -= a[i][j] * x[j];
        }
        x[i] /= a[i][i];
    }
}

static void parallel_gaussian(double **a, int n, int num_threads, double *x)
{
    pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
    thread_data_t *thread_data = malloc(num_threads *
sizeof(thread_data_t));
    pthread_barrier_t barrier;
```

```
    pthread_barrier_init(&barrier, NULL, num_threads);

    for (int k = 0; k < n - 1; k++) {
        for (int i = 0; i < num_threads; i++) {
            thread_data[i].thread_id = i;
            thread_data[i].num_threads = num_threads;
            thread_data[i].n = n;
            thread_data[i].a = a;
            thread_data[i].k = k;
            thread_data[i].barrier = &barrier;

            pthread_create(&threads[i], NULL, eliminate_rows,
&thread_data[i]);
        }

        for (int i = 0; i < num_threads; i++) {
            pthread_join(threads[i], NULL);
        }
    }

    pthread_barrier_destroy(&barrier);
    free(threads);
    free(thread_data);

    for (int i = n - 1; i >= 0; i--) {
        x[i] = a[i][n];
        for (int j = i + 1; j < n; j++) {
            x[i] -= a[i][j] * x[j];
        }
        x[i] /= a[i][i];
    }
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Использование: %s <размер матрицы N> <количество
потоков>\n", argv[0]);
        return 1;
    }

    char *endptr1;
    char *endptr2;
    int n = (int)strtol(argv[1], &endptr1, 10);
    int num_threads = (int)strtol(argv[2], &endptr2, 10);
```

```
    if (*endptr1 != '\0' || *endptr2 != '\0' || n <= 0 || num_threads <= 0)
{
        printf("Некорректные аргументы командной строки\n");
        return 1;
    }

    double **a_seq = allocate_matrix(n);
    double **a_par = allocate_matrix(n);
    double *x_seq = malloc(n * sizeof(double));
    double *x_par = malloc(n * sizeof(double));

    generate_matrix(a_seq, n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= n; j++) {
            a_par[i][j] = a_seq[i][j];
        }
    }

    struct timespec start;
    struct timespec end;

    clock_gettime(CLOCK_MONOTONIC, &start);
    sequential_gaussian(a_seq, n, x_seq);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double seq_time = (double)(end.tv_sec - start.tv_sec) * 1000.0 +
(double)(end.tv_nsec - start.tv_nsec) / 1000000.0;

    clock_gettime(CLOCK_MONOTONIC, &start);
    parallel_gaussian(a_par, n, num_threads, x_par);
    clock_gettime(CLOCK_MONOTONIC, &end);
    double par_time = (double)(end.tv_sec - start.tv_sec) * 1000.0 +
(double)(end.tv_nsec - start.tv_nsec) / 1000000.0;

    printf("N = %d, Threads = %d\n", n, num_threads);
    printf("Seq Time: %.2f ms\n", seq_time);
    printf("Par Time: %.2f ms\n", par_time);
    printf("Speedup:  %.2f\n", seq_time / par_time);
    printf("Efficiency: %.2f\n", (seq_time / par_time) / num_threads);

    free_matrix(a_seq, n);
    free_matrix(a_par, n);
    free(x_seq);
    free(x_par);
```

```
    return 0;
}
```

Протокол работы программы

Ускорение (S):

- **Назначение:** Показывает, во сколько раз параллельная программа работает быстрее последовательной.
- **Как вычислить:** Разделить время выполнения последовательной программы на время выполнения параллельной:

$$S = \frac{T_{seq}}{T_{par}}$$

- **Пояснение переменных:** T_{seq} - время выполнения в 1 потоке (мс), T_{par} - время выполнения в N потоках (мс).

Эффективность (E):

- **Назначение:** Показывает степень использования вычислительных ресурсов процессора (насколько эффективно каждый поток расходует свое время).
- **Как вычислить:** Разделить ускорение на количество потоков:

$$E = \frac{S}{N}$$

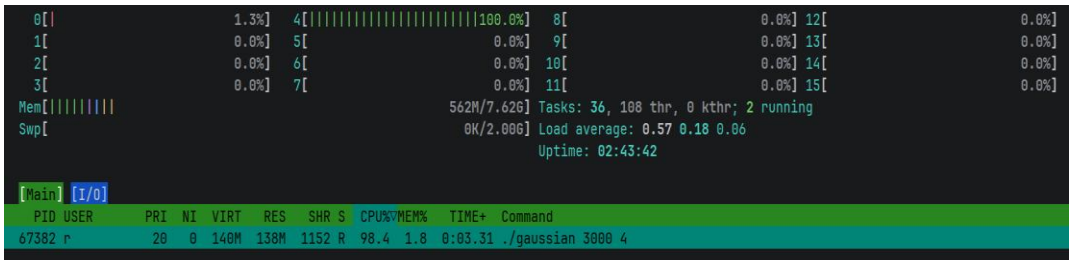
- **Пояснение переменных:** S - полученное ускорение, N - количество запущенных потоков.

Таблица тестов при N = 3000:

Число потоков (N)	Время исполнения	Ускорение (S)	Эффективность (E)	Описание режима

	(Tpar , мс)			
1	5344.25	0.96	0.96	Последовательный режим
2	3443.43	1.57	0.79	Меньше ядер
4	2761.54	1.96	0.49	Меньше ядер
6	3659.58	1.49	0.25	Меньше ядер
8	4238.73	1.26	0.16	Меньше ядер
16	4580.62	1.22	0.08	Равно числу логических ядер
128	13152.46	0.41	0.00	Сильно больше ядер
1024	96222.06	0.05	0.00	Критический перегруз потоков

```
r@DESKTOP-N828GQK:~/labs/os-labs/lab2$ ./gaussian 3000 4
N = 3000, Threads = 4
Seq Time: 5674.15 ms
Par Time: 3611.49 ms
Speedup:  1.57
Efficiency: 0.39
```



Вывод

В ходе выполнения лабораторной работы были приобретены практические навыки создания, управления и синхронизации потоков в операционной системе с использованием библиотеки **POSIX Threads** (pthreads). На языке Си была реализована параллельная версия алгоритма решения СЛАУ методом Гаусса с использованием системного вызова pthread_barrier_t для синхронизации потоков на каждом шаге прямого хода.

На основе проведенных экспериментов при N = 3000 сформирована таблица замеров и сделаны следующие ключевые выводы:

1. **Оптимальный режим:** Наилучшее время выполнения (2761.54 мс) и максимальное ускорение (S = 1.96) были достигнуты при N = 4 потоках.

2. **Падение эффективности при перегрузе:** При дальнейшем увеличении числа потоков (6, 8, 16) и особенно при критическом перегрузе (128 и 1024 потока) время исполнения резко увеличилось до 96222.06 мс, а ускорение упало до 0.05. Это обусловлено высокими накладными расходами операционной системы на планирование, переключение контекста и синхронизацию барьерами между огромным количеством потоков, соперничающих за ограниченные ресурсы процессора.

3. **Эффективность ресурсопользования:** Максимальная эффективность (E = 0.79 ... 0.96) наблюдается на 1 ... 2 потоках. При запуске сотен потоков эффективность стремится к 0.00, что наглядно демонстрирует нецелесообразность создания числа потоков, значительно превышающего количество доступных логических ядер системы.

В процессе работы возникла проблема с зависанием при запуске 1024 потоков из-за колоссальной нагрузки на планировщик ОС, которая была решена оптимизацией запускных параметров. Поставленная задача выполнена в полном объёме.